A Report

On

DIGITAL DOOR LOCK SYSTEM

Submitted

By

Sl. No.	Student Name	Student ID
1	Pujan Chordiya	2022A3PS1296H
2	Soham Erande	2022AAPS1376H
3	Garv Agarwal	2022A8PS1307H

UNDER THE SUPERVISION OF

PROF. ANAKHI HAZARIKA

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

HYDERABAD CAMPUS

(November, 2024)

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to the EEE Department of BITS Pilani, Hyderabad Campus, for providing me with the opportunity to work on the project titled "Digital Door Lock System”. This project would not have been possible without the support and encouragement of our esteemed faculty members.

Above all, I would like to express my sincere gratitude to Prof. Anakhi Hazarika for her wise counsel, constant support, and invaluable mentoring during this endeavor. Her insightful counsel and helpful criticism were extremely important in determining the course and effective completion of this effort. Many of the difficulties faced throughout the research were solved in large part because of her practical insights and subject-matter competence.

Lastly, I would like to express my gratitude to everyone of the EEE Department's staff, instructors, and other students that helped this project get done and progress. Their support and eagerness to help made this project a really worthwhile educational opportunity.

CONTENT

Topics	Page No.
Abstract	4
Aim of the Project	5
Importance of the Project	5
System Architecture	6
Components Overview	8
Facial Recognition	11
Interfacing Camera	13
PIN-Based Authentication	13
RFID Master Key Override	15
Servo Motor	20
Source Code	21
Future Work	22
References	23

ABSTRACT

This project presents the design and implementation of a digital door lock system based on Raspberry Pi 5 developed as part of the “Embedded System Design” course. The system provides a reliable yet flexible access control system with the aid of multi-level security provisions. Face recognition has been chosen as the means of unlocking the door by making use of the inbuilt face recognition software that has a machine learning classification application behind it. In cases where face recognition does not work, a fallback approach has been created that anticipates the user to enter a pin number on a 3x4 keypad. When both of these methods fail, access can only be allowed by presenting an RFID card, which is treated as a master key. The working of a face recognition system, a keypad and an RFID sensor interfaced with a Raspberry Pi 5 is a clear illustration of embedded systems at work. The project emphasizes on the relevant dimensions of hardware interface, machine learning and design, thus yielding an effective and flexible security system.

AIM OF THE PROJECT

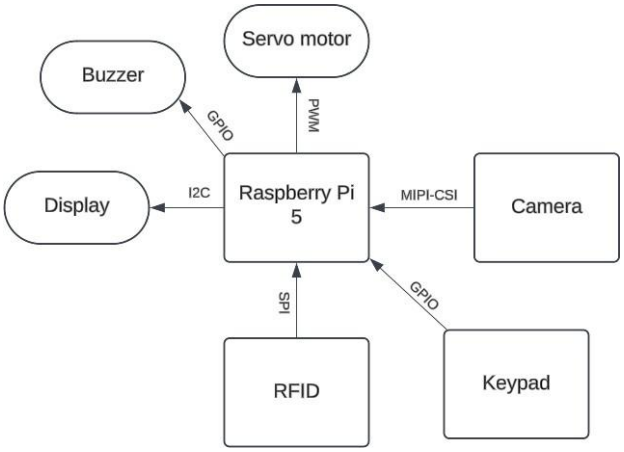
This project proposes to develop and enhance a high level security system using Raspberry Pi 5, with ultimate focus on face recognition for unlocking the door, supported by 3x4 keypad for entering pins and RFID sensor for master key access. This system is an illustration of embedded system design principles applied in practice in the real world security solutions field.

IMPORTANCE OF THE PROJECT

In today's world, security and convenience are top priorities and for that, the development of advanced locking systems is necessary for protecting personal and organizational assets. In this project, we integrate multiple authentication modes for better security with face recognition biometric measures and secondary authentication through PIN entry and RFID access. All integrated on Raspberry Pi 5, a cost-effective, adaptable, and efficient platform is provided for real-world applications in homes or offices. Furthermore, this project bridges theoretical learning with practical implementation, enabling us to gain hands-on experience in embedded systems and hardware interfacing. This project highlights the critical place of multi-modal authentication systems in providing a more trusted and user-friendly solution to the security problem.

SYSTEM ARCHITECTURE

Block diagram



The digital lock system relies on effective communication among the different peripherals and Raspberry Pi 5 which incorporate relevant interfacing protocols. To interface the Raspberry Pi Camera, which is used for face recognition, is connected via the MIPI CSI-2 (Camera Serial Interface 2) protocol. MIPI CSI-2 is termed as the open high performance point-to-point interface for reliable video data communication, satisfying all accuracy and fluency requirements in facial recognition systems.

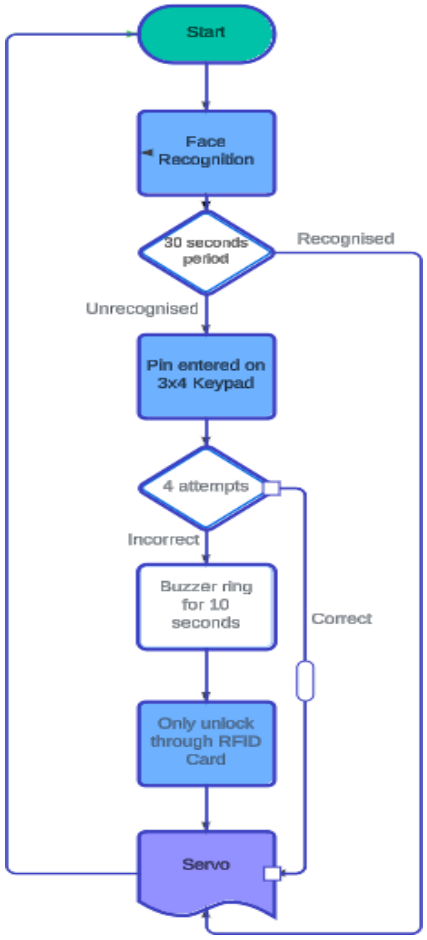
The SPI (Serial Peripheral Interface) protocol is specified to communicate with the RFID sensor. SPI is a time-tested protocol for synchronous serial communication with fast and reliable data transmission between the Raspberry Pi and the RFID module. Its simple and high-speed operation makes it suitable for implementation for this purpose.

For facial recognition, we have employed deep learning, a subset of machine learning which uses convolutional neural networks(CNNs) and facial recognition library which employs facenet architecture, consisting of 22+ convolutional layers. Not only is this library responsible for facial recognition but also for feature extraction, thus making it more robust. There are 128-Dimensional Embeddings which represent the facial features.

Also, the 3x4 keypad and buzzer are connected through the GPIO pins of the Raspberry Pi. Using GPIO pins for interfacing eliminates the need for an intermediate controller, simplifying the design while providing reliable input detection through

row-column scanning. Such protocols integrated can ensure seamless performance of all those hardware components and indeed showcase the flexibility, versatility and adaptability of the Raspberry Pi 5 with respect to using it in various embedded system applications.

System Functionality Flowchart



The system starts by attempting face recognition for 30 seconds.If the face is recognized, the system proceeds to the "Serve" action.If the face is not recognized, the user is prompted to enter a PIN on a 3x4 keypad.The user has 4 attempts to enter the correct PIN.If the PIN is incorrect after 4 attempts, a buzzer sounds for 10 seconds, and the system can only be unlocked using an RFID card.

Components overview

1. **Raspberry Pi 5**
    - The digital door locking system's central processing unit, the Raspberry Pi 5, offers the processing power and interface capabilities required to effectively manage several authentication methods. With notable performance and adaptability enhancements, it is the most recent model in the Raspberry Pi family..
    - It is powered by a 2.4 GHz quad-core ARM Cortex-A76 processor.
    - Memory (RAM) - Machine learning techniques for facial recognition may be handled with ease thanks to the 4GB LPDDR4X RAM variations.
    - Connectivity- Bluetooth 5.0 and Wi-Fi 6 are integrated for wireless connection, opening the possibility of future remote access or monitoring.
    - 40 General Purpose Input/Output (GPIO) pins for interfacing with peripherals like the camera, keypad, RFID module, and servo motor.
    - Runs Raspberry Pi OS, a lightweight Linux-based operating system optimized for the hardware.
    - Supports Python, C++, and other programming languages for coding the system’s logic.

2. **Camera Module for Facial Recognition** (Raspberry Pi Camera)
    - The Raspberry Pi Camera Rev 1.3 is a small, lightweight camera module intended for Raspberry Pi devices. It has a 5-megapixel OV5647 sensor that can record HD video at 1080p30, 720p60, or 640x480p60/90 and take excellent still photos at 2592x1944 resolution. The module ensures effective data transfer by connecting directly to the Raspberry Pi via the CSI (Camera Serial Interface) port. It is perfect for activities involving computer vision, such as facial recognition, photography, and video recording. It is appropriate for embedded projects like door lock systems

and home automation setups due to its plug-and-play nature and tiny form factor.

3. **3x4 Keypad for PIN Entry**
    - The 3x4 matrix keypad is used as the secondary authentication mechanism.
    - It provides 12 keys (0-9 digits, * and #) for secure PIN entry.
    - Easily interfaced with the Raspberry Pi through GPIO pins, the keypad adds a reliable fallback option in case facial recognition fails.

4. **RFID Sensor and Master Key Tag** (RC522)
    - For RFID-based applications, the RC522 RFID module provides a flexible and affordable option. It can be used with secure access control and identification systems because it operates at 13.56 MHz and supports ISO/IEC 14443A standard RFID cards and tags. By integrating an MFRC522 IC, the module makes it possible to read and write RFID data efficiently. It ensures compatibility and versatility by interacting with microcontrollers, such the Raspberry Pi, through SPI interface. The module is a great option for embedded systems and Internet of Things

projects because of its small size, low power consumption, and onboard antenna for dependable detection within a short range.

- Module Interface SPI Data Transfer Rate: Max. 10Mbit/s
- Card reading distance：0～30mm（Mifare1 card）

5. **Servo Motor**
    - A servo motor is employed to control the door lock mechanism.
    - It offers precise angular movement, allowing the lock to open and close securely based on authentication success.
    - The servo is directly controlled by the Raspberry Pi through PWM signals.

6. **Power Supply and Other Peripherals**
    - The system is powered by a Type-C power adapter, ensuring a stable and sufficient power supply for the Raspberry Pi 5 and connected components.
    - Additional peripherals include jumper wires, resistors, and connectors required for interfacing and assembly.

This selection of components is cost-effective and efficient, ensuring robust functionality and smooth operation of the digital door locking system.

Facial Recognition

Facial recognition is done using the camera integrated with the raspberry pi. When the correct face is recognised continuously for 5 seconds, the lock will open. If the correct face is not recognised continuously for 5 seconds in the span of 30 seconds after the first face is detected, the lock can only be opened by entering the PIN or using RFID.

Working Principle:

We have used the facenet architecture, which makes use of convolutional neural networks, which identify the edges of a person and thus detect him/her. Facenet is a special architecture, designed and coded for facial recognition and makes use of the following things:

(1) Deep learning convolutional neural networks.
(2) 22+ convolutional layers.
(3) 128-dimensional vector for each face image, capturing its unique features.
(4) Low Memory Requirement
(5) Invariant to Variations: FaceNet learns robust embeddings that are invariant to:
- Pose variations.
- Illumination changes.
- Occlusions (e.g., glasses, hats).

(6) Adding new faces doesn't require retraining the network.

This means, even if the person is wearing or not wearing spectacles/sunglasses/hat,etc. The person will be detected.

In this model, we only give 1 image as input for each person. So, if we want 3 people to be detected, we have to input 3 images, 1 per person.

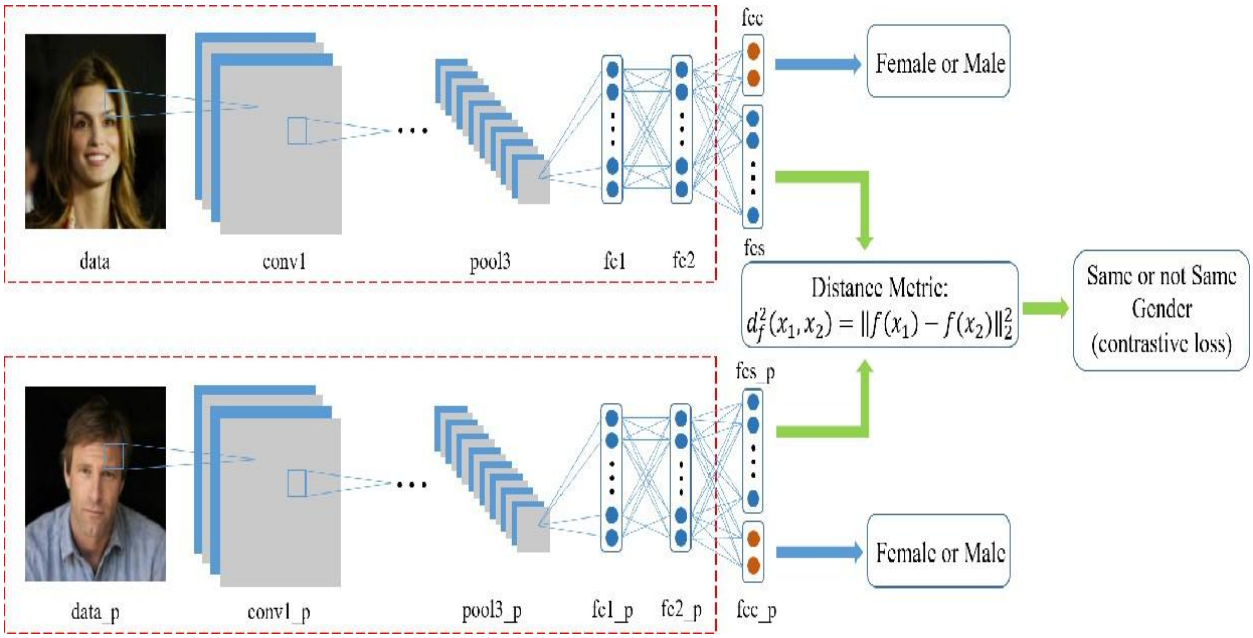
Once an image is detected, the architecture creates a compact 128-dimensional vector array that uniquely represents that face and every time a face comes in front of the camera, such a vector array is generated and this array is matched with the previous array of the image we have already fed in. We also define certain thresholds such that the model is very strict w.r.t detections, with a high threshold(may result in a few false negatives) or very lenient w.r.t. Detections, with a low threshold(may result in a few false positives). Based on these threshold values, the values in these vectors are matched and the model predicts whether the person in front of the camera is the person in the photo fed to the model.
Since feature extraction is used in this model, it makes the model very much robust and better as compared to the other existing models for facial recognition like YOLO.

This architecture uses only around 22 convolutional layers, as compared to the 53 convolutional layers used in YOLO, making the model very fast and lighter to run on hardware.

Since everything is run on Raspberry pi only, none of the data is cloud sourced or even connected to the internet, thus making the model very secure from cyber attacks as well.

Performance:

- Achieved confidence levels of between 0.65-0.95 in the detection with 0 false detections.
- 100% precise and accurate in detections.
- Resulting delay for image processing close to 0 seconds.
- Excellent performance on images outside the dataset.



Interfacing Camera for Facial Recognition

Facial recognition is performed on the feed that is provided by the Raspberry Pi Camera Rev1.3.
The camera is interfaced with the Raspberry Pi 5 using the dedicated camera and display connectors present on the board and using the appropriate ribbon cables. The camera communicates with the Raspberry Pi 5 using MIPI CSI - 2 protocol where CSI stands for Camera Serial Interface and MIPI stands for Mobile Industry Processor Interface. MIPI CSI-2 is a serial communication protocol and is widely used in the majority of embedded camera and imaging interfaces.
For the facial recognition part, we preprocess the image dataset generating the appropriate encoding pickle file. This processing of the dataset to get the pickle file is done on the RaspberryPi 5 itself . Now we run the model using facenet library and extract image frames from the camera using opencv library. We appropriately process the image and then run the model on the image to get the facial recognition. The pickle file contains the labels for the authorized person which were there in the dataset and model then gives us the probability that the face in the camera is the same as the person in the dataset. If the probability threshold is exceeded then a bounding box will be drawn around the recognized face and a subsequent name label will be added. If the face is unrecognized the label will simply change to unknown.

Code for interfacing:
Click here for the code file

PIN-Based Authentication

PIN entering is done using a 3x4 matrix keypad with 12 buttons stacked in rows and columns. When a key is pressed, the keypad recognizes the relationship between a row and a column. The following procedures was followed in order to interface it with the Raspberry Pi:

- The keypad has 7 pins: 3 for rows and 4 for columns.
- Connected the **row pins** (R1, R2, R3) to 3 GPIO pins on the Raspberry Pi.
- Connected the **column pins** (C1, C2, C3, C4) to 4 other GPIO pins.

Since the Raspberry Pi GPIO pins are used to both read and output signals, the rows will be configured as output pins, and the columns will be input pins.

Working Principle & Interfacing:

1. **Output Stage (Rows)**: The Raspberry Pi sends a HIGH (3.3V) signal to one row at a time.
2. **Input Stage (Columns)**: The Raspberry Pi monitors the column pins, which are connected to the individual keys. If a key is pressed, it creates a path between a row pin and a column pin.
3. The program detects this connection, identifies the specific row and column, and determines which key was pressed.

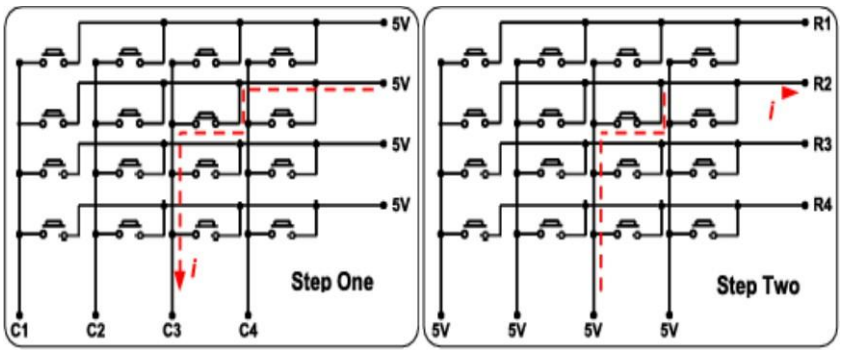


Fig:- Keypad (4x4) working principle

Code for interfacing:

Click here for the code file

Explanation of code:

It first initializes the GPIO pins; the rows will be configured as output, while the columns will be inputs with a pull-down resistor to detect key presses. The keypad layout is defined in a 2D list called keyPad, where each key corresponds to a number or symbol.

The read_keypad() function scans each row, setting it HIGH one at a time, and checks for a HIGH signal in any column. When a key is pressed, it returns the corresponding value from the keyPad matrix. A debounce delay of 0.3 seconds ensures that key presses are detected accurately.

It asks the user to enter a 4-digit passcode. If they input a correct key sequence that matches the predefined passcode, then it prints "Unlocked." If incorrect, it asks them to try again. The GPIO is cleaned up at the end of the program or on keyboard interrupt.

RFID Master Key Override

By identifying and confirming the UID of an approved RFID tag, the master key override is put into practice. It will be activated once all other methods for opening the lock are failed.

Working Principle & interfacing:

Radio Communication: RC522 uses electromagnetic field communication. In this mode, the reader transmits a 13.56 MHz signal through its antenna, which is received by RFID tags within its range. These tags are generally passive, without a power source and simply on the energy from a reader's signal.

Tag Activation : Upon entering the RF field, the RFID tag comes alive through the electromagnetic waves. The tag then communicates with the system by sending back a specific UID or other data stored in its chip through modulation of the electromagnetic field.

The reception and processing of data takes place when the RC522 detects the tag's response and decodes transmitted data. This data is then relayed to a microcontroller through an interface like SPI (Serial Peripheral Interface), which then analyses the received data which might be for purposes such as door unlocking, logging information, or sending to a database.

Read and Write Operations: Besides reading tags' data, the RC522 can be used to write data into writable RFID tags, though this feature would depend on which type of tag is in use. The module can be used with different types of RFID tag formats such as MIFARE cards.

The RC522 does not have its own large storage memory but interacts with RFID tags, which themselves have a predefined memory structure. The most commonly used RFID cards with the RC522 are MIFARE Classic cards, which have particular memory layouts.

1. MIFARE Classic Card Memory Layout

A normal MIFARE 1K card has 1024 bytes of total memory divided into 16 sectors, each sector containing 4 blocks (each block is 16 bytes).

Sectors: There are 16 sectors in a MIFARE 1K card, from 0 to 15.

Sector 0: This usually contains manufacturer data and the access key for the rest of the sectors.
Sector 1 to 15: These sectors are for storing user data; e.g., serial numbers and authentication data.
Blocks: Each sector is divided into 4 blocks. Data is stored in the first 3 blocks of every sector, and the 4th block is dedicated to the sector trailer, where access keys A and B are provided to manage the contents of the memory.

The Key Structure: Each sector contains two keys, A and B, which may be used with authentication before reading or writing data in that sector. The key length is typically 6 bytes.

2. Memory Segments in MIFARE 1K:

Block 0: Data from the manufacturer (UID and other information)
Block 1–3: Can be utilized for data or authentication key storage.
Sector Trailer: Block 3 of each sector holds the access control keys, Key A and Key B, which permit or deny entry to other blocks in the sector.

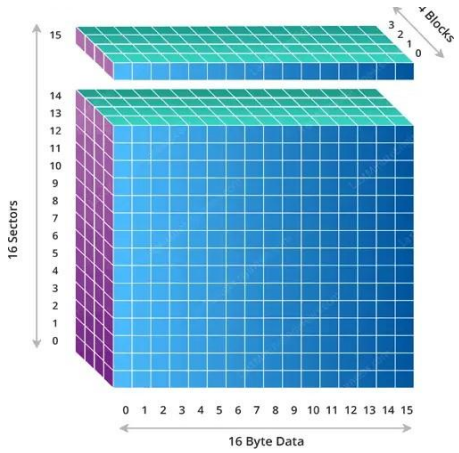


Fig: MIFARE 1K memory layout diagram.

3. SPI interfacing with Raspberry Pi:

SPI Communication Setup: The RC522 module is the SPI slave and the Raspberry Pi is the SPI master. The Raspberry Pi sends commands via the MOSI pin and sets the SDA pin low (chip select) to initiate communication.

Power Up and Initialization: The RC522 starts initializing and awaits a command from the Raspberry Pi as soon as it is powered on. Using particular registers, the Raspberry Pi configures the SPI interface and puts the RC522 module into an operational mode.

Card Detection: To identify RFID cards, the RC522 continually sends out radio waves through its antenna. The RC522 immediately recognizes a card and initiates communication as soon as it is within range.

Card Reading: This RFID module being based on MF RC522 is a close-range communicator that uses the 13.56 MHz signal to transmit the information to RFID cards (MIFARE Classic). The RC522 being activated receives the UID (Unique Identifier) and other data from the card when the card approaches within the range of the reader. To initiate communication, the RC522 to the card sends an REQA or WUPA command which replies with its UID.
After receiving the data into the FIFO, the module operates as a part of internal memory where information is buffered for transmission. Read or write of data on the card is permitted only after the key is authenticated with the help of a key contained in the sector trailer (Block 3 of each sector).
This module connects to the Raspberry Pi utilizing SPI (Serial Peripheral Interface). Through SPI, the master is the Raspberry Pi, while the RC522 serves as the slave. The command is sent to the CS (*chip select*) pin to start the data transfer. The data is passed to the MISO (Master In Slave Out) pin by the FIFO register in a serial manner during the clock pulse generated on the SCK (Serial Clock) pin of the Raspberry Pi. The data are transmitted to the various registers for management and control by the RF identifications. These registers-including the FIFO register (0x09), Command register (0x01), and Status register (0x2A)-control the data flow, monitor the operations, and indicate whether they were successful. The Raspberry Pi reads the data received from the MISO pin byte by byte, processes it, and finally is able to perform higher functions such as reading other blocks or writing data to the card.

Writing Data: To write data to an RFID card using the RC522 module, the steps followed are essentially the same as for reading. However, the process will include the authentication step besides transferring data through the card's memory. Upon writing a suitable selectable card, the RC522 performs an access control on the selected sector using Key A or Key B, which are found within block 3 of the card sector trailer. During authenticating a card, the RC522 sends an AUTHENT command to the card via SPI, alongside the required key for authentication.

Upon successful authentication, the RC522 can write data onto the card. The data will be sent to the FIFO register of the RC522, where it is buffered before dispatch. The

data will be sent by the Raspberry Pi to be written to the card through the SPI interface. The data is sent from the Raspberry Pi to the RC522 via the MOSI pin, where it is then going to be written to the memory block chosen by the card.

The RC522 manages the writing through its various internal registers, such as the command register (0x01) and FIFO register (0x09), controlling the flow of the data to ensure proper writing on the card. After the data is written, it verifies the operation's success by checking the Status register (0x2A). The data is written in a memory block of the card, thus updating its internal data.

Once inserting data is done, the communication is disabled, while Raspberry Pi can perform other tasks. The basic features of that process enable RC522 to write data into RFID cards in a reliable way, with proper authentication, as well as error checking.

Code for interfacing:

Click Here for code file

Explanation of code:

This Python code communicates with a Raspberry Pi-connected RFID module. It communicates with the RFID reader and controls GPIO pins using the RPi.GPIO and MFRC522 libraries.

Libraries:

RPi.GPIO: This library enables communication with the GPIO pins of the Raspberry Pi. Input/output pin configuration, state change (HIGH/LOW), and value reading are all made possible by it. An LED is controlled by this code.

The MFRC522 RFID reader is a widely used RFID module that can read and write RFID tags. This library communicates with it. A streamlined interface for interacting with the reader and retrieving RFID data is offered by the library.

time: Time-related functions, such sleep(), which pauses the application for a predetermined amount of seconds, are provided by the time module.

Code Dissection:

1. **GPIO Configuration**:
    a. GPIO.setwarnings(False) turns off GPIO pin use warnings.
    b. The pin numbering is adjusted using GPIO.setmode(GPIO.BOARD) to match the actual pin arrangement on the Raspberry Pi board.
    c. Pin 40 is configured as an output pin for the LED via GPIO.setup(led, GPIO.OUT).
    d. The LED is first turned off (LOW state) thanks to GPIO.output(led, GPIO.LOW).
2. **Initialization of the Reader:**
    a. The RFID reader object is created with reader = SimpleMFRC522().
3. **Principal Loop:**
    a. The application uses reader.read() to continuously scan for RFID tags. The ID of the tag and the related text data that is stored on the RFID tag are the two values that this method returns. The LED attached to pin 40 turns on for one second if the RFID tag's ID matches a predetermined value (872589727). The LED stays off if they don't match.
4. **Handling Errors:**
    a. In order to avoid conflicts, the unless block makes sure that any configured GPIO resources are relinquished correctly by calling GPIO.cleanup() in the event of an error.
5. **Operation of Functions:**
    a. reader.read(): After reading the RFID tag, this function delivers the tag's unique ID along with any stored text.
    b. GPIO.output(): To power the LED, it regulates the HIGH or LOW state of a GPIO pin.

Note: We had used LED for to check for interfacing of RFID and hence code contains LED as output but in actual, servo motor is mode of output.

Servo Motor

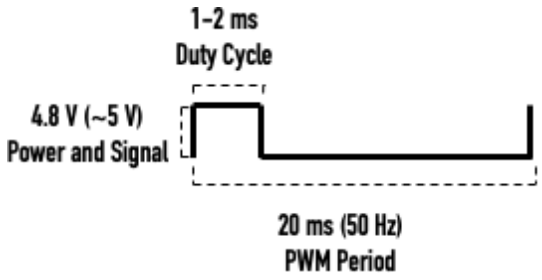
The servo motor is used as a type of actuator which corresponds to the action of opening of the lock. The servo motor moves from 0 degree position to 90 degree position and then again back to the 0 degree position when the correct face is recognised or the correct pin is entered or the RFID masterkey is detected.

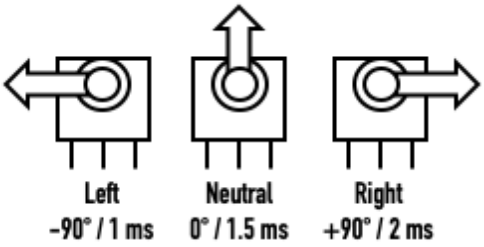
Interfacing:

The servo motor has 3 leads red, black and yellow corresponding to power(Vcc), ground and signal respectively. The SG90 servo that we have used needs the Vcc to supply 5V regulated supply which is handled by the 5V pins on the Raspberry Pi. The signal pin is connected to one of the GPIO pins on the Raspberry Pi.

Working:

The SG90 Servo motor has a signal wire which receives PWM signals of 50Hz (20ms) and the duty cycle of the signal should vary between 1ms and 2ms, 1ms signifying 0 degree or -90 degrees and 2ms signifying 180 degree or +90 degrees. Other angles are then assumed by linearly extrapolating between 1ms and 2ms. The servo receives the angle input at 50Hz and is able to update its position at 50Hz as well.





Code for interfacing:

Click Here for code file

Code explanation:

- First, we import the RPi.GPIO library and the sleep function.

  ```
  import RPi.GPIO as GPIO
  from time import sleep
  ```

- Then we setup the GPIO mode as BOARD so that we can reference the PINs and not the BCM pins.

  ```
  GPIO.setmode(GPIO.BOARD)
  GPIO.setup(11, GPIO.OUT)
  ```

- We connect the servo motor to pin 11 on the Raspberry Pi.
- Next, we create a variable for the servo called "PWM." Then, send a 50 Hz PWM signal on that GPIO pin using the GPIO.PWM() function. Start the signal at 0.

  ```
  pwm=GPIO.PWM(11, 50)
  pwm.start(0)
  ```

We use the ChangeDutyCycle() function to write duty cycle percentages to the servo motor. We use the three duty cycle percentages calculated at the beginning of this step. (Left -90 deg is 5%, Neutral is 7.5%, and Right +90 deg is 10%.) Give the servo 1 second to reach its position.

```
pwm.ChangeDutyCycle(5) # left -90 deg position
sleep(1)
pwm.ChangeDutyCycle(7.5) # neutral position
sleep(1)
pwm.ChangeDutyCycle(10) # right +90 deg position
sleep(1)
```

Lastly, clean up the code by stopping the PWM signal and running the cleanup function on the GPIO pins.

```
pwm.stop()
GPIO.cleanup()
```

Source Code

Click Here for the integrated source code.

Future Work

The global smart door lock market size was valued at USD 10.34 billion in 2023. The market is projected to grow from USD 11.23 billion in 2024 to USD 32.7 billion by 2030, exhibiting a CAGR of 17% during the forecast period:. Main reason for this is biometric authentication methods, such as fingerprint recognition and facial recognition. These authentication mechanisms offer higher security by accurately identifying individuals based on unique biological traits. Furthermore, the COVID-19 pandemic accelerated the adoption of smart door locks as people sought contactless solutions for their home security. With an increased focus on hygiene and touchless interactions, smart locks gained traction for their convenience and safety features. However, there is always room for improvement and innovation. Here are some potential areas for future work on smart door locks:

- **Making a multilayer system:** Implementing a single layer security system we have now with multiple that is 2 factor authentication like having both facial recognition and passkey for more security.
- **Biometric Authentication:** Improving the accuracy and speed of biometric authentication like fingerprint recognition, facial recognition, and introducing iris scanning.
- **AI-Powered Threat Detection:** Implement AI algorithms to detect unusual behavior or attempted break-ins, triggering alerts and recording suspicious activities.
- **Easy Installation:** Developing systems that are easy to install without assistance of professionals using manuals and videos
- **Battery Life Optimization:** Research ways to extend battery life, such as energy-efficient hardware and software optimization.
- **Multi-User:** Creating multi-user system, allowing owner to easily grant and remove access for other users.
- **Wearable Device Integration:** Allowing them to be controlled by wearable devices like smartwatches and fitness trackers with the help of RFID
- **Energy Source:** Powering smart locks using energy sources like solar power or kinetic energy from door usage.

By addressing these areas, future smart door locks can become more secure, user-friendly, and integrated into the broader ecosystem of smart home and IoT devices.

References

1. Face Recognition With Raspberry Pi and OpenCV - Tutorial Australia
2. How to Control a Servo with Raspberry Pi - Learn Robotics
3. How to Interface RFID-RC522 With Raspberry Pi - Instructables
4. how to install yolo v8 on raspberry pi bookworm 64 bit & create your own custom model yolov8
5. FaceNet - Using Facial Recognition System - GeeksforGeeks
6. Facial Recognition for Raspberry Pi with OpenCV and Python (Updated Tut…
7. ATIF AFROZ (2022), “Digital Smart Door Lock Security System Using Arduino Uno Microcontroller”, Gurukul kangri University, Haridwar, India, Available at: https://www.irejournals.com/formatedpaper/1703609.pdf
8. Felix Gunawan, Chih-Lyang Hwang, and Zih-En Cheng. (2023), “ROI-YOLOv8-Based Far-Distance Face-Recognition”, Available at: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10268512